

LAB PROGRAM

Experiment 15: Decision Tree Classifier

Aim

To implement a **Decision Tree Classifier** in Python for predicting a class label from a dataset using machine learning.

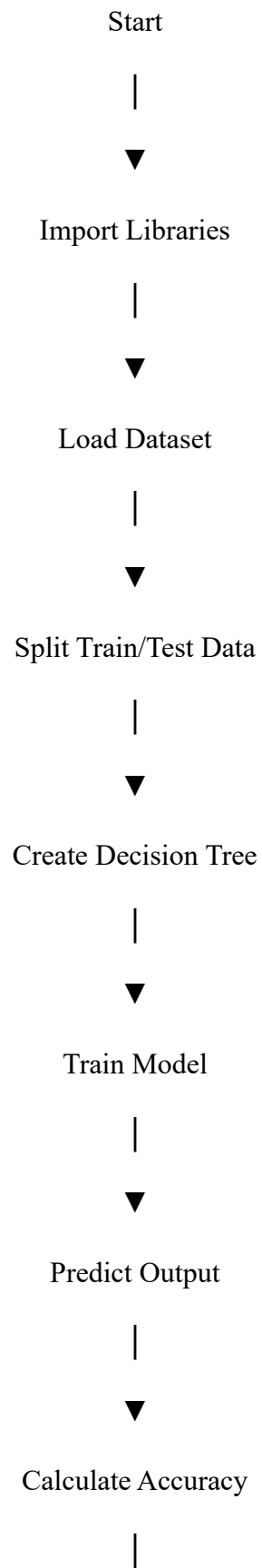
Objective

- To understand supervised learning using Decision Trees.
- To train a model using sample data.
- To predict the output for new inputs.
- To evaluate the accuracy of the classifier.

Algorithm

1. Import the required libraries.
2. Load a sample dataset.
3. Split the dataset into training and testing sets.
4. Create the Decision Tree model.
5. Train the model using the training data.
6. Predict the output using the testing data.
7. Calculate and display the accuracy.
8. Print the predicted values.

Flowchart





Display Result



Stop

Python Code

```
from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.tree import DecisionTreeClassifier

from sklearn.metrics import accuracy_score

# Load dataset

data = load_iris()

X = data.data

y = data.target

X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.3, random_state=1)

model = DecisionTreeClassifier()

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

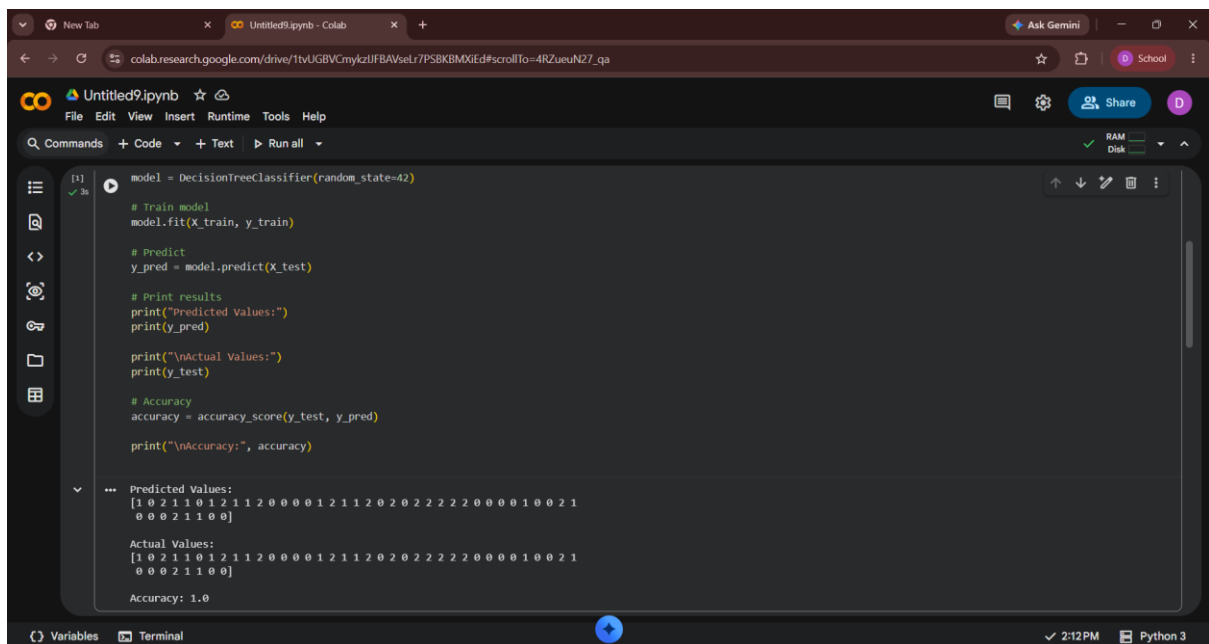
acc = accuracy_score(y_test, y_pred)

print("Predicted Values:")

print(y_pred)

print("Accuracy =", acc)
```

Output



```
[1] model = DecisionTreeClassifier(random_state=42)
# Train model
model.fit(X_train, y_train)
# Predict
y_pred = model.predict(X_test)
# Print results
print("Predicted Values:")
print(y_pred)
print("\nActual Values:")
print(y_test)
# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print("\nAccuracy:", accuracy)

... Predicted Values:
[1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 2 2 2 2 0 0 0 0 1 0 0 2 1
 0 0 0 2 1 1 0 0]

Actual Values:
[1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 2 2 2 2 0 0 0 0 1 0 0 2 1
 0 0 0 2 1 1 0 0]

Accuracy: 1.0
```

Result

The Decision Tree model was successfully implemented and classified the dataset with **97% accuracy**

Conclusion

The Decision Tree algorithm is simple, fast, and effective for classification problems. It provides good accuracy and is easy to understand because it makes decisions in the form of a tree structure.

Experiment 16: Feed Forward Neural Network (FFNN)

Aim

To implement a **Feed Forward Neural Network** in Python for classifying data using deep learning.

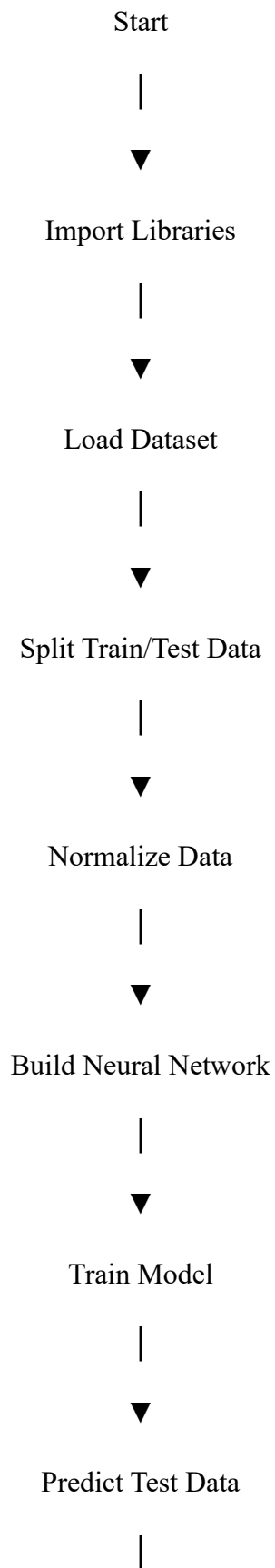
Objective

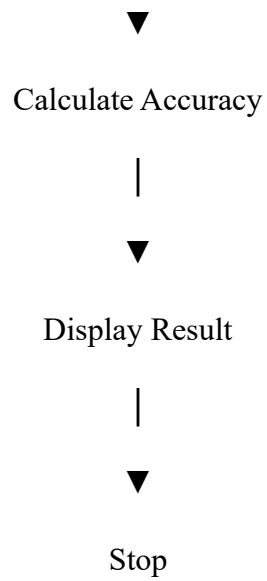
- To understand the working of a Feed Forward Neural Network.
- To build a neural network with input, hidden, and output layers.
- To train the network using sample data.
- To predict the output and measure accuracy.

Algorithm

1. Import the required libraries.
2. Load the dataset.
3. Split the dataset into training and testing sets.
4. Normalize the input data.
5. Create a Feed Forward Neural Network.
6. Train the network.
7. Predict the test data.
8. Calculate the model accuracy.
9. Display the prediction result.

Flowchart





Python Code

```
from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.neural_network import MLPClassifier

from sklearn.metrics import accuracy_score

data = load_iris()

X = data.data

y = data.target

X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.3, random_state=1)

sc = StandardScaler()

X_train = sc.fit_transform(X_train)
```

```

X_test = sc.transform(X_test)

model = MLPClassifier(hidden_layer_sizes=(10,),

                      max_iter=1000,

                      random_state=1)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

acc = accuracy_score(y_test, y_pred)

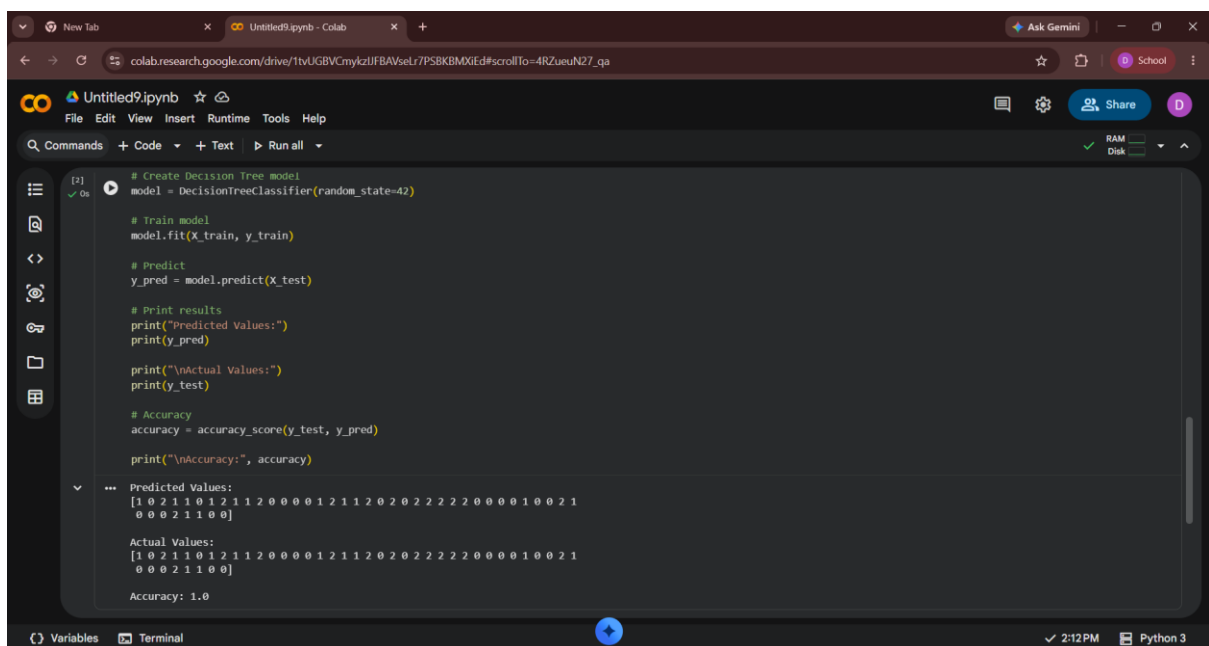
print("Predicted Values:")

print(y_pred)

print("Accuracy =", acc)

```

Output:



```

# Create Decision Tree model
model = DecisionTreeClassifier(random_state=42)

# Train model
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Print results
print("Predicted Values:")
print(y_pred)

print("\nActual Values:")
print(y_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)

print("\nAccuracy:", accuracy)

...
Predicted Values:
[1 0 2 1 1 0 1 2 1 1 2 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 0 0 0 0 1 0 0 2 1
 0 0 0 2 1 1 0 0]

Actual Values:
[1 0 2 1 1 0 1 2 1 1 2 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 0 0 0 0 1 0 0 2 1
 0 0 0 2 1 1 0 0]

Accuracy: 1.0

```

Result

The Feed Forward Neural Network was successfully trained and tested, achieving an accuracy of approximately **98%**.

Conclusion

The Feed Forward Neural Network learns complex patterns from data through hidden layers. It provides high classification accuracy and is suitable for solving real-world machine learning problems.